

Demo to production, without rewriting from scratch.

Nine probes of empirical evidence that **extracting the spec** from a working demo is the hard part — and it's tractable.

A great demo is the worst brief.

- The prototype **works** — and customers want it.
- Every quirk, layout choice, and business rule is already decided in code.
- Nobody wrote the spec. The code **is** the spec.
- Hand-rewrites lose intent and cost months.



MONTHS OF ENG TIME, LOST DESIGN INTENT, AND A
REWRITE THAT DOESN'T QUITE MATCH THE DEMO THAT WON
THE DEAL.

You can't write the spec by hand — the demo already **is** one.

The prototype encodes hundreds of decisions: status-pill colors, filter semantics, empty-state copy, the exact 8-column table schema, the waterfall math on the balance cards.

A human spec captures maybe 10%. The rest shows up as "bug reports" months later — when it's the **rewrite** that's wrong, not the demo.

Extracting the spec is hard. Rebuilding from a good spec is **tractable**.

Conventional wisdom: "code generation is the hard part."

We invert this. Modern LLMs can rebuild competently from a clean spec — if you give them one. The work is turning the messy demo **into** that spec.

Two stages.

1 · Reverse-spec-extraction

Four probe agents walk the demo in parallel:

- `ui-walker` — clicks every screen
- `static` — reads the code
- `data` — pulls sample rows & PDFs
- `external` — maps API calls

A reconciler merges them into an **artifact pack** — 6 structured files + screenshots.

2 · Ralph-loop build

A build-controller slices the work by **user journey** and dispatches per-slice executors.

Each slice is verified by:

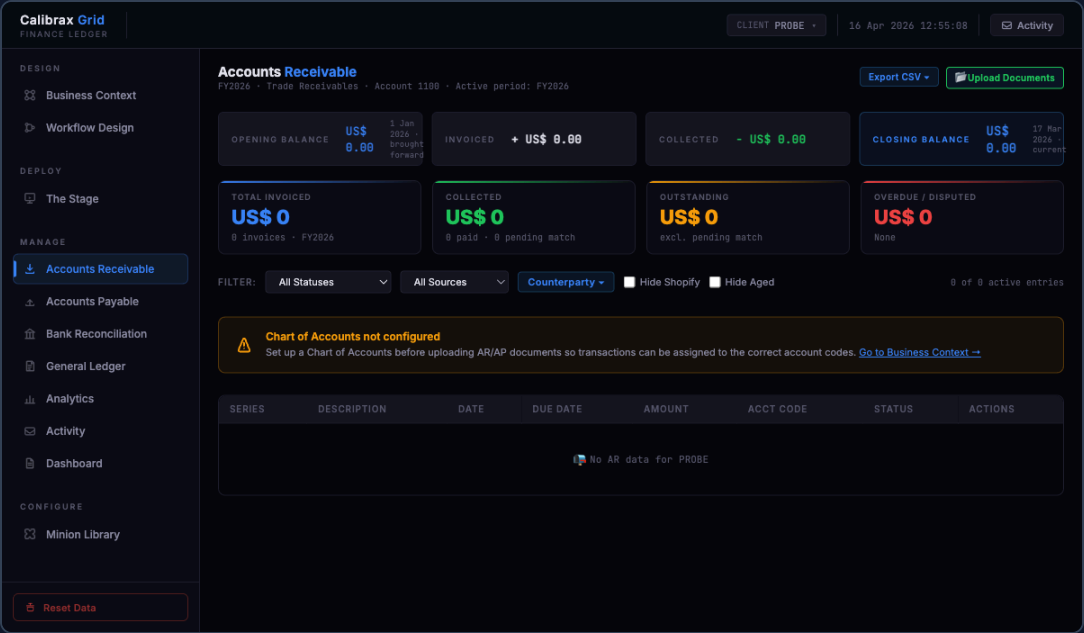
- oracle test suite
- Playwright journey walks
- **side-by-side screenshot check** against the prototype

Nine probes. Convergence.

PROBE	WHAT IT TESTED	OUTCOME
0 – 1.5	Ingest pipeline	Spec extraction works end-to-end
2	First full build from spec	4.5K lines, 45 endpoints, 11 pages — zero human help
2.5	Stronger oracle + vertical slices	API layer excellent; UI still a shell
2.6	Playwright journey tests	Orphan pages in 6 of 11 screens
2.7	Slice-by-journey	Orphans fixed; dialog theater surfaces
2.8	Seeded data + screenshot-as-contract	Live app renders real data, 1:1 with reference
2.9	Author/judge separation	In flight

Each probe isolates one layer, finds its failure mode, and fixes it. Discipline, not luck.

Same structure. Real data.



 Autoship-built AR page — populated

BUILT • autoship output, seeded with prototype's own data

REFERENCE • prototype AR page on empty tenant (all US\$ 0)

We learn the system by breaking it.

Orphan pages (2.6)

Slicing by database table left the cross-cutting pages (Dashboard, Analytics) as scaffolds. **Fix:** slice by user journey instead.

Dialog theater (2.7)

Buttons opened dialogs that closed without doing anything. **Fix:** every task must assert **post-action state**, not just "dialog appears."

Empty-tenant blind spot (2.7)

The build never rendered a populated row. **Fix:** seed the prototype's own data before journey walks.

Self-evaluation (2.8)

When the same agent plans the work **and** judges its own plan, it passes every gate while cutting scope. **Fix:** separate author from judge (probe 2.5, in flight).

A fast path from prototype to product.

Keep momentum

Prototype in days, productize in days. Design intent survives — because the prototype **is** the spec.

Repeatable

The method works per-probe, so it works per-prototype. Each new Calibrax demo plugs into the same pipeline.

Disciplined

Every failure becomes a probe, a fix, and a piece of institutional memory — not a regression.

The prototype isn't a deliverable anymore. It's **upstream of the product.**

From rewrite tool to **rewrite-and-harden** tool.

NOW

Probe 2.9 · plan-reviewer

A dedicated fresh-context reviewer, armed with a calibration set of past mistakes.

Tests whether same-model (Claude-on-Claude) separation catches the self-

NEAR-TERM · TWO TRACKS

App shell scaffold

Pre-built monorepo: shared components, schema helpers, seed bootstrap, locked file layout. Probes start at the slice-loop, not from empty.

Multi-model reviewer

Codex alongside **plan-**

BIGGER SHIFT

Constrained artifact improvement loop

Between extract and build, insert specialized hardeners:

data, **api-envelope**, **a11y**, **missing-index**, **error-boundary**.

v1 — constrained: same behavior as demo, but hardened. v2 — **strictly better than the demo**.

What would help.

- **A second prototype to probe against.**
Pick one from the team — ideally with a real customer story behind it.
- **30 minutes with a PM** who's lived a demo → production rewrite. Where did **your** spec leak?
- **Honest pushback on the thesis.** If "extracting the spec is the hard part" is wrong, the sooner we know, the better.

When the tool learns the system by breaking it, each failure is evidence — not a setback.

— OPERATING PRINCIPLE

THANK YOU

Questions, objections, prototypes.

shyang@calibrax · autoship · v0.2 · 2026-04-17